

## JavaScript Metaprogramming — Symbols, Proxy & Reflect শেখার মূল উদ্দেশ্য

JavaScript-এ অবজেক্টের স্বাভাবিক আচরণ পরিবর্তন, অপারেশন ইন্টারসেপ্ট ও কাস্টম লজিক যুক্ত করার আর্কিটেকচারাল মেকানিজমের নাম হলো **Metaprogramming**। এই চ্যাপ্টারে আমরা **Unique Symbol Keys**, **Global Symbol Registry**, **Proxy Handler Traps (get, set, has, deleteProperty)**, **Reflect API** এবং রিঅ্যাক্টিভ ফ্রেমওয়ার্ক আর্কিটেকচার নিখুঁতভাবে শিখব।

- |                             |                             |                            |
|-----------------------------|-----------------------------|----------------------------|
| ★ 1. Primitive Symbol       | ★ 2. Unique Property Keys   | ★ 3. Symbol.for() Registry |
| ★ 4. Symbol.keyFor()        | ★ 5. Well-Known Symbols     | ★ 6. Symbol.iterator       |
| ★ 7. Symbol.toPrimitive     | ★ 8. Proxy Target & Handler | ★ 9. get & set Traps       |
| ★ 10. Schema Validation     | ★ 11. has & deleteProperty  | ★ 12. apply & construct    |
| ★ 13. Proxy.revocable       | ★ 14. Reflect Static API    | ★ 15. Reflect.get/set      |
| ★ 16. Reflect.ownKeys       | ★ 17. Proxy + Reflect Pair  | ★ 18. Read-Only Objects    |
| ★ 19. Access Control System | ★ 20. Vue Reactivity Engine | ★ 21. 5 Common Traps       |
| ★ 22. 9 Practice Challenges |                             |                            |

### Part 01 — Symbols & Unique Property Keys (28.1 – 28.12)

#### 28.1 Symbol কী?

Symbol হলো JavaScript-এর একটি **primitive data type**। JavaScript-এর primitive types:

##### ► CONSOLE OUTPUT

##### OUTPUT

```
String
Number
BigInt
Boolean
Undefined
Null
Symbol
```

Symbol তৈরি করা যায়:

##### JAVASCRIPT SOURCE

##### JAVASCRIPT

```
const id = Symbol();
```

## 28.2 Symbol কেন দরকার?

ধরো দুটি property একই নামের:

```
JAVASCRIPT SOURCE JAVASCRIPT
const user = {
  id: 101
};

user.id = 202;
```

আগের value replace হয়ে যাবে। কিন্তু Symbol ব্যবহার করলে unique key তৈরি করা যায়।

```
JAVASCRIPT SOURCE JAVASCRIPT
const id1 = Symbol();
const id2 = Symbol();

console.log(id1 === id2);
```

Output:

```
► CONSOLE OUTPUT OUTPUT
false
```

প্রতিটি Symbol সাধারণত unique।

## 28.3 Symbol-এর Real-Life Idea

ধরো একটি বড় application-এ অনেক developer কাজ করছে। একজন লিখল:

```
JAVASCRIPT SOURCE JAVASCRIPT
user.status
```

আরেকজনও লিখল:

```
JAVASCRIPT SOURCE JAVASCRIPT
user.status
```

Property collision হতে পারে। Symbol:

```
JAVASCRIPT SOURCE JAVASCRIPT
const internalStatus = Symbol();

user[internalStatus] = "active";
```

এতে একটি unique key তৈরি হলো।

## 28.4 Symbol-এর Type

```
JAVASCRIPT SOURCE JAVASCRIPT
const id = Symbol();

console.log(typeof id);
```

Output:

```
► CONSOLE OUTPUT OUTPUT
symbol
```

### 28.5 Symbol Description

Symbol তৈরি করার সময় description দেওয়া যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const id = Symbol("userId");  
  
console.log(id);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Symbol(userId)
```

Description মূলত debugging-এর জন্য helpfull

### 28.6 Description Access

JAVASCRIPT SOURCE

JAVASCRIPT

```
const id = Symbol("userId");  
  
console.log(id.description);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
userId
```

### 28.7 Symbol Always Unique

JAVASCRIPT SOURCE

JAVASCRIPT

```
const a = Symbol("id");  
const b = Symbol("id");  
  
console.log(a === b);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
false
```

যদিও description একই:

► CONSOLE OUTPUT

OUTPUT

```
id
```

তবুও Symbol দুটো আলাদা।

### 28.8 Symbol as Object Property

```
JAVASCRIPT SOURCE JAVASCRIPT

const userId = Symbol("userId");

const user = {
  name: "Shariar",
  [userId]: 101
};

console.log(user.name);
console.log(user[userId]);
```

Output:

```
► CONSOLE OUTPUT OUTPUT

Shariar
101
```

খেয়াল করো:

```
JAVASCRIPT SOURCE JAVASCRIPT

[userId]
```

computed property syntax ব্যবহার করা হয়েছে।

### 28.9 Dot Notation দিয়ে Symbol Access করা যায় না

এটা ভুল:

```
JAVASCRIPT SOURCE JAVASCRIPT

user.userId
```

কারণ `userId` এখানে Symbol variable নয়। সঠিক:

```
JAVASCRIPT SOURCE JAVASCRIPT

user[userId]
```

### 28.10 Symbol Property কি Hidden?

Symbol property **private** নয়। এটা খুব গুরুত্বপূর্ণ।

```
JAVASCRIPT SOURCE JAVASCRIPT

const secret = Symbol("secret");

const user = {
  name: "Shariar",
  [secret]: "Hello"
};
```

`Object.keys()` করলে:

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log(Object.keys(user));
```

Output:

```
► CONSOLE OUTPUT OUTPUT

["name"]
```

Symbol key এখানে আসে না। কিন্তু:

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log(Object.getOwnPropertySymbols(user));
```

Output:

```
► CONSOLE OUTPUT OUTPUT

[Symbol(secret)]
```

অর্থাৎ: > Symbol property hidden from some normal enumeration, কিন্তু private/security mechanism নয়।

### 28.11 Reflect.ownKeys()

String এবং Symbol—দুই ধরনের own keys পেতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const id = Symbol("id");

const user = {
  name: "Shariar",
  [id]: 101
};

console.log(Reflect.ownKeys(user));
```

Output-এর ধারণা:

► CONSOLE OUTPUT

OUTPUT

```
["name", Symbol(id)]
```

### 28.12 Symbol বনাম String Key

| String Key            | Symbol Key                                                         |
|-----------------------|--------------------------------------------------------------------|
| "id"                  | Symbol("id")                                                       |
| collision হতে পারে    | unique                                                             |
| Object.keys()-এ আসে   | Object.keys()-এ আসে না                                             |
| সাধারণ property       | special property key                                               |
| সাধারণভাবে enumerable | Symbol property-ও enumerable হতে পারে, কিন্তু Object.keys-এ আসে না |

## Part 02 — Global Symbol Registry & Well-Known Symbols (28.13 – 28.21)

### 28.13 Symbol.for()

এখন একটি গুরুত্বপূর্ণ বিষয়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const a = Symbol("user");
const b = Symbol("user");

console.log(a === b);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
false
```

কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const a = Symbol.for("user");
const b = Symbol.for("user");

console.log(a === b);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
true
```

কারণ `Symbol.for()` **Global Symbol Registry** ব্যবহার করে।

#### 28.14 Symbol.for() কীভাবে কাজ করে?

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.for("app.user")
```

প্রথমবার তৈরি হলে registry-তে রাখে। আবার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.for("app.user")
```

করলে একই Symbol পাওয়া যায়। Mental model:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Symbol("user")
  ↓
নতুন unique Symbol
Symbol.for("user")
  ↓
Global Registry
  ↓
একই key হলে একই Symbol
```

#### 28.15 Symbol.keyFor()

Symbol.for() দিয়ে তৈরি Symbol-এর key বের করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const userSymbol = Symbol.for("user");
console.log(Symbol.keyFor(userSymbol));
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

user

#### 28.16 Symbol.for() বনাম Symbol()

| Symbol()                   | Symbol.for()              |
|----------------------------|---------------------------|
| নতুন Symbol তৈরি করে       | Registry থেকে Symbol নেয় |
| প্রতিবার unique            | একই key → একই Symbol      |
| Global registry নয়        | Global Symbol Registry    |
| Symbol.keyFor() কাজ করে না | Symbol.keyFor() কাজ করে   |

#### 28.17 Well-Known Symbols

JavaScript কিছু built-in Symbol দিয়েছে যেগুলো language behavior customize করতে সাহায্য করে। Examples:

▶ CONSOLE OUTPUT

OUTPUT

```
Symbol.iterator
Symbol.toStringTag
Symbol.toPrimitive
Symbol.toStringTag
Symbol.hasInstance
Symbol.asyncIterator
Symbol.species
Symbol.isConcatSpreadable
```

সবচেয়ে গুরুত্বপূর্ণ কয়েকটি আমরা দেখব।

## 28.18 Symbol.iterator

Chapter 27-এ আমরা এটা দেখেছি। এটি কোনো object-এর iteration behavior define করতে পারে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = {

  *[Symbol.iterator]() {

    yield 10;
    yield 20;
    yield 30;

  }

};

for (const number of numbers) {
  console.log(number);
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
10
20
30
```

এখানে:

JAVASCRIPT SOURCE

JAVASCRIPT

Symbol.iterator

object-টাকে iterable করেছে।

## 28.19 Symbol.toStringTag

Object-এর `Object.prototype.toString()` result customize করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {

  [Symbol.toStringTag]: "User"

};

console.log(Object.prototype.toString.call(user));
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
[object User]
```

## 28.20 Symbol.toPrimitive

Object primitive value-তে convert হওয়ার সময় behavior customize করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const money = {  
  value: 100,  
  [Symbol.toPrimitive](hint) {  
    if (hint === "number") {  
      return this.value;  
    }  
    return `$$${this.value}`;  
  }  
};  
  
console.log(+money);  
console.log(`${money}`);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
100  
$100
```

এটি advanced metaprogramming-এর example।

## 28.21 Symbol.hasInstance

instanceof behavior customize করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
class EvenNumber {  
  static [Symbol.hasInstance](value) {  
    return typeof value === "number" && value % 2 === 0;  
  }  
}  
  
console.log(2 instanceof EvenNumber);  
console.log(3 instanceof EvenNumber);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
true  
false
```

এখানে instanceof -এর behavior custom করা হয়েছে।

### 🔥 Symbol-এর সবচেয়ে গুরুত্বপূর্ণ ব্যবহার

Practical JavaScript-এ তোমার জন্য সবচেয়ে গুরুত্বপূর্ণ:

► CONSOLE OUTPUT

OUTPUT

1. Unique property keys
2. Avoid property collisions
3. Symbol.iterator
4. Well-known Symbols
5. Metaprogramming

**28.22** এখন আসি Proxy-তে

Proxy হলো JavaScript-এর একটি powerful feature যেটা কোনো object-এর operations **intercept** এবং **customize** করতে পারে। সহজ ভাষায়: > Proxy হলো object-এর সামনে একটি "middle layer"। Mental model:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Code
↓
Proxy
↓
Target Object
```

**28.23** Basic Proxy

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {});

console.log(proxy.name);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

Shariar

এখানে:

▶ CONSOLE OUTPUT

OUTPUT

```
user = target
proxy = wrapper
```

**28.24** Proxy Handler

Proxy তৈরি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
new Proxy(target, handler)
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar"
};

const handler = {
  get(target, property) {
    console.log(`Reading: ${property}`);
    return target[property];
  }
};

const proxy = new Proxy(user, handler);

console.log(proxy.name);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
Reading: name
Shariar
```

### 28.25 get Trap

get trap property access intercept করে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar",
  age: 23
};

const proxy = new Proxy(user, {

  get(target, property) {

    console.log("Accessing:", property);

    return target[property];

  }

});

console.log(proxy.name);
console.log(proxy.age);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Accessing: name
Shariar

Accessing: age
23
```

### 28.26 set Trap

Property set করার সময় intercept করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {

  set(target, property, value) {

    console.log(`Setting ${property} = ${value}`);

    target[property] = value;

    return true;

  }

});

proxy.name = "Ripon";
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Setting name = Ripon
```

## 28.27 Proxy দিয়ে Validation

এটি খুব practical।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {  
  name: "Shariar",  
  age: 23  
};  
  
const proxy = new Proxy(user, {  
  set(target, property, value) {  
    if (property === "age") {  
      if (typeof value !== "number") {  
        throw new TypeError("Age must be a number");  
      }  
  
      if (value < 0) {  
        throw new RangeError("Age cannot be negative");  
      }  
    }  
  
    target[property] = value;  
  
    return true;  
  }  
});  
  
proxy.age = 25;  
  
console.log(proxy.age);
```

Output:

► CONSOLE OUTPUT

OUTPUT

25

কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
proxy.age = -10;
```

Error:

► CONSOLE OUTPUT

OUTPUT

RangeError: Age cannot be negative

## 28.28 has Trap

`in` operator intercept করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {
  has(target, property) {
    console.log(`Checking: ${property}`);
    return property in target;
  }
});

console.log("name" in proxy);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Checking: name
true
```

## 28.29 deleteProperty Trap

`delete` operation intercept করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar",
  age: 23
};

const proxy = new Proxy(user, {
  deleteProperty(target, property) {
    console.log(`Deleting: ${property}`);
    return delete target[property];
  }
});

delete proxy.age;

console.log(user);
```

### 28.30 ownKeys Trap

`Object.keys()` /reflection-related key retrieval customize করা যায়।

```
JAVASCRIPT SOURCE JAVASCRIPT

const user = {
  name: "Shariar",
  age: 23
};

const proxy = new Proxy(user, {

  ownKeys(target) {
    return ["name"];
  }

});

console.log(Object.keys(proxy));
```

Output:

```
► CONSOLE OUTPUT OUTPUT

["name"]
```

এখানে `age` hide করা হয়েছে enumeration থেকে।

### 28.31 Proxy Logging System

Proxy দিয়ে property access log করা যায়।

```
JAVASCRIPT SOURCE JAVASCRIPT

const user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka"
};

const loggedUser = new Proxy(user, {

  get(target, property) {
    console.log(
      `GET → ${String(property)}`
    );
    return target[property];
  },

  set(target, property, value) {
    console.log(
      `SET → ${String(property)} = ${value}`
    );
    target[property] = value;
    return true;
  }

});

console.log(loggedUser.name);

loggedUser.age = 24;
```

এটি debugging বা development tooling-এ useful হতে পারে।

### 28.32 apply Trap

Proxy শুধু object-এর property operation intercept করে না। Function-এর invocation-ও intercept করতে পারে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function greet(name) {  
  return `Hello ${name}`;  
}  
  
const proxy = new Proxy(greet, {  
  apply(target, thisArg, args) {  
    console.log("Function called");  
    return target(...args);  
  }  
});  
  
console.log(proxy("Shariar"));
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Function called  
Hello Shariar
```

### 28.33 construct Trap

new operation intercept করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
const ProxyUser = new Proxy(User, {  
  construct(target, args) {  
    console.log("Creating User");  
    return new target(...args);  
  }  
});  
  
const user = new ProxyUser("Shariar");
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Creating User
```

### 28.34 Proxy Trap Summary

সব trap মুখস্থ করার দরকার নেই। Important:

| Trap           | Intercepts     |
|----------------|----------------|
| get            | property read  |
| set            | property write |
| has            | in             |
| deleteProperty | delete         |
| ownKeys        | keys           |
| apply          | function call  |
| construct      | new            |

### 28.35 Proxy + Private Fields

এখানে একটি গুরুত্বপূর্ণ limitation আছে। Private class fields:

JAVASCRIPT SOURCE

JAVASCRIPT

```
#password
```

Proxy দিয়ে সাধারণ property-এর মতো intercept করা যায় না।

JAVASCRIPT SOURCE

JAVASCRIPT

```
class User {  
  
  #password = "12345";  
  
}
```

Private field-এর access language-level private mechanism দ্বারা controlled। তাই Proxy-কে security boundary হিসেবে ব্যবহার করা উচিত নয়।

### 28.36 Proxy Revocable

একটি Proxy temporarily disable/revoke করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {  
  name: "Shariar"  
};  
  
const { proxy, revoke } = Proxy.revocable(user, {});  
  
console.log(proxy.name);  
  
revoke();  
  
console.log(proxy.name);
```

শেষ access-এর সময় error হবে।

## Part 04 — Reflect API & Metaprogramming (28.37 – 28.45)

### 28.37 এখন আসি Reflect-এ

Reflect হলো JavaScript-এর built-in object যেটা অনেক **object operation-এর standard methods** দেয়। Examples:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Reflect.get()  
Reflect.set()  
Reflect.has()  
Reflect.deleteProperty()  
Reflect.ownKeys()  
Reflect.construct()  
Reflect.apply()
```

### 28.38 Reflect.get()

সাধারণ:

```
JAVASCRIPT SOURCE JAVASCRIPT
user.name
```

Reflect:

```
JAVASCRIPT SOURCE JAVASCRIPT
Reflect.get(user, "name");
```

Example:

```
JAVASCRIPT SOURCE JAVASCRIPT
const user = {
  name: "Shariar"
};

console.log(
  Reflect.get(user, "name")
);
```

Output:

```
► CONSOLE OUTPUT OUTPUT
Shariar
```

### 28.39 Reflect.set()

```
JAVASCRIPT SOURCE JAVASCRIPT
const user = {};

Reflect.set(user, "name", "Shariar");

console.log(user.name);
```

Output:

```
► CONSOLE OUTPUT OUTPUT
Shariar
```

### 28.40 Reflect.has()

```
JAVASCRIPT SOURCE JAVASCRIPT
const user = {
  name: "Shariar"
};

console.log(
  Reflect.has(user, "name")
);
```

Output:

```
► CONSOLE OUTPUT OUTPUT
true
```

এটি conceptually:

```
JAVASCRIPT SOURCE JAVASCRIPT
"name" in user
```

এর মতো।

#### 28.41 Reflect.deleteProperty()

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar",
  age: 23
};

Reflect.deleteProperty(user, "age");

console.log(user);
```

#### 28.42 Reflect.ownKeys()

String + Symbol keys:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const id = Symbol("id");

const user = {
  name: "Shariar",
  [id]: 101
};

console.log(
  Reflect.ownKeys(user)
);
```

এটি দুই ধরনের key দিতে পারে:

► CONSOLE OUTPUT

OUTPUT

```
"name"
Symbol(id)
```

#### 28.43 Reflect কেন Proxy-এর সাথে বেশি দেখা যায়?

এটি খুব গুরুত্বপূর্ণ। Proxy handler-এ manually:

JAVASCRIPT SOURCE

JAVASCRIPT

```
target[property] = value;
```

করার পরিবর্তে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Reflect.set(target, property, value);
```

ব্যবহার করা অনেক ক্ষেত্রে safer এবং cleaner।

## 28.44 Proxy + Reflect

Recommended pattern:

```
JAVASCRIPT SOURCE JAVASCRIPT

const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {

  get(target, property, receiver) {

    console.log(`GET: ${String(property)}`);

    return Reflect.get(
      target,
      property,
      receiver
    );
  },

  set(target, property, value, receiver) {

    console.log(
      `SET: ${String(property)} = ${value}`
    );

    return Reflect.set(
      target,
      property,
      value,
      receiver
    );
  }
});

console.log(proxy.name);

proxy.name = "Ripon";
```

## Proxy + Reflect Mental Model

```
🕒 ARCHITECTURE / CONCEPT FLOW

Code
↓
Proxy
↓
Trap
↓
Reflect
↓
Target Object
```

অর্থাৎ:

```
🕒 ARCHITECTURE / CONCEPT FLOW

Proxy = intercept
Reflect = perform standard operation
```

## 28.45 Metaprogramming কী?

**Metaprogramming** মানে এমন code লেখা যা program-এর behavior/structure-এর ওপর কাজ করতে পারে বা customize করতে পারে। JavaScript-এ:

```
🕒 ARCHITECTURE / CONCEPT FLOW

Proxy
Reflect
Symbols
Property Descriptors
```

এসব metaprogramming-এর tools।

**28.46 Real-Life Example: Read-Only Object**

Proxy দিয়ে property পরিবর্তন block করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const settings = {
  theme: "dark",
  language: "en"
};

const readOnlySettings = new Proxy(settings, {
  set() {
    throw new Error(
      "Settings are read-only"
    );
  }
});

console.log(readOnlySettings.theme);

readOnlySettings.theme = "light";
```

শেষ line-এ error হবে।

**28.47 Real-Life Example: Access Control**

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar",
  role: "user",
  salary: 50000
};

const safeUser = new Proxy(user, {
  get(target, property) {
    if (property === "salary") {
      if (target.role !== "admin") {
        return "Access denied";
      }
    }
    return Reflect.get(target, property);
  }
});

console.log(safeUser.name);
console.log(safeUser.salary);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
Shariar
Access denied
```

⚠ এটি demonstration। Real application security-এর জন্য server-side authorization অবশ্যই প্রয়োজন।

## 28.48 Reactive Systems-এর সাথে Connection

Proxy modern frontend architecture-এর কিছু concept বুঝতে সাহায্য করে। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const state = {  
  count: 0  
};
```

Proxy দিয়ে change detect:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const reactiveState = new Proxy(state, {  
  set(target, property, value) {  
    console.log(  
      `${property} changed`  
    );  
    return Reflect.set(  
      target,  
      property,  
      value  
    );  
  }  
});  
  
reactiveState.count = 1;
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
count changed
```

এই ধরনের concept reactive systems/framework internals বোঝার ক্ষেত্রে গুরুত্বপূর্ণ।

## 28.49 Symbol + Proxy একসাথে

Symbol দিয়ে internal-like metadata রাখা যায়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const metadata = Symbol("metadata");  
  
const user = {  
  name: "Shariar",  
  [metadata]: {  
    createdBy: "system"  
  }  
};  
  
const proxy = new Proxy(user, {  
  get(target, property) {  
    if (property === metadata) {  
      return Reflect.get(target, property);  
    }  
    return Reflect.get(target, property);  
  }  
});
```

এখানে Symbol unique key হিসেবে কাজ করছে।

### 28.50 Proxy Performance Consideration

Proxy powerful হলেও সব জায়গায় ব্যবহার করা উচিত নয়। কারণ proxy layer:

🕒 ARCHITECTURE / CONCEPT FLOW

```
property access
  ↓
trap
  ↓
target
```

অতিরিক্ত abstraction তৈরি করতে পারে। তাই performance-sensitive code-এ অপ্রয়োজনীয় Proxy ব্যবহার avoid করা ভালো।

### 28.51 Common Mistake: Symbol-কে Private ভাবা

ভুল ধারণা: > Symbol মানেই private property! ❌ না। Symbol property discover করা যায়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Object.getOwnPropertySymbols(obj)
```

অথবা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Reflect.ownKeys(obj)
```

তাই Symbol: > **Unique property key**, security/privacy mechanism নয়।

### 28.52 Common Mistake: Symbol("id") একই হবে ভাবা

JAVASCRIPT SOURCE

JAVASCRIPT

```
const a = Symbol("id");
const b = Symbol("id");

console.log(a === b);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
false
```

একই description ≠ same Symbol।

### 28.53 Common Mistake: Proxy Target Automatically Protected

Proxy বানালেই original object protected হয় না।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {});

user.name = "Ripon";
```

Original target সরাসরি পরিবর্তন করা সম্ভব।

### 28.54 Common Mistake: set Trap-এ Boolean Return

set trap-এর ক্ষেত্রে সাধারণত success indicate করতে boolean return করতে হয়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
set(target, property, value) {
  target[property] = value;

  return true;
}
```

## 28.55 Common Mistake: Proxy-কে Security Boundary ভাবা

Proxy দিয়ে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
salary access block  
password property hide
```

করলেও client-side JavaScript-এর ওপর security নির্ভর করা উচিত নয়। Real security:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Frontend  
↓  
Backend Authorization  
↓  
Database
```

## Part 06 — Production Mastery, Practice Lab & Mental Map (28.56 – 28.59)

### 28.56 MUST KNOW

#### ◆ Symbol

JAVASCRIPT SOURCE

JAVASCRIPT

```
const id = Symbol("id");
```

#### ◆ Symbol Property

JAVASCRIPT SOURCE

JAVASCRIPT

```
obj[id] = 100;
```

#### ◆ Global Symbol

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.for("id");
```

#### ◆ Well-known Symbol

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.iterator
```

#### ◆ Proxy

JAVASCRIPT SOURCE

JAVASCRIPT

```
new Proxy(target, handler);
```

#### ◆ Get trap

JAVASCRIPT SOURCE

JAVASCRIPT

```
get(target, property)
```

#### ◆ Set trap

JAVASCRIPT SOURCE

JAVASCRIPT

```
set(target, property, value)
```

#### ◆ Reflect

JAVASCRIPT SOURCE

JAVASCRIPT

```
Reflect.get()  
Reflect.set()  
Reflect.has()  
Reflect.deleteProperty()  
Reflect.ownKeys()
```

#### ◆ Most important relationship

🕒 ARCHITECTURE / CONCEPT FLOW

```
Proxy → intercept  
Reflect → perform operation  
Symbol → customize/identify
```

## 28.Cheat 🍷 Quick Cheat Sheet — Symbol, Proxy & Reflect Matrix

Metaprogramming-এর ৩টি মূল স্তরের সংক্ষিপ্ত ব্যবহারিক সিনট্যাক্স সামারি:

### 1. Symbol Syntax

| SYMBOL REFERENCE | JAVASCRIPT |
|------------------|------------|
|------------------|------------|

```
const s = Symbol("desc");
const g = Symbol.for("app.id");
Symbol.keyFor(g); // 'app.id'
obj[Symbol.iterator] = ...;
```

### 2. Proxy Syntax

| PROXY PATTERN | JAVASCRIPT |
|---------------|------------|
|---------------|------------|

```
const proxy = new Proxy(target, {
  get(target, prop, receiver) {
    return
  },
  set(target, prop, val, receiver) {
    return
  }
});
```

### 3. Reflect Syntax

| REFLECT METHODS | JAVASCRIPT |
|-----------------|------------|
|-----------------|------------|

```
Reflect.get(target, prop);
Reflect.set(target, prop, val);
Reflect.has(target, prop);
Reflect.deleteProperty(target, p);
Reflect.ownKeys(target);
```

## 28. Lab 📄 Hands-On Practice Lab — 9 Production Metaprogramming Challenges

Symbol, Proxy Traps এবং Reflect API সম্পূর্ণ আয়ত্তে আনতে নিচে ৩টি লেভেলের ৯টি প্র্যাকটিক্যাল চ্যালেঞ্জের সম্পূর্ণ মডেল সলিউশন দেওয়া হলো:

### Practice 1: Symbol as Object Property Key

BEGINNER

**Problem Requirement:** একটি `userId` সিম্বল তৈরি করে এবং অবজেক্টের প্রোপার্টি হিসেবে ব্যবহার করে তা কনসোলে অ্যাক্সেস করো।

#### PRACTICE 1 MODEL SOLUTION

✓ STANDARD PATTERN

```
const userId = Symbol("userId");

const user = {
  name: "Ripon",
  [userId]: 1001
};

console.log("Name:", user.name);
console.log("Symbol ID:", user[userId]);
console.log("Object Keys:", Object.keys(user)); // Symbol excluded
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
Name: Ripon
Symbol ID: 1001
Object Keys: ['name']
```

### Practice 2: Symbol Uniqueness Verification

BEGINNER

**Problem Requirement:** একই বিবরণ (description) দিয়ে দুটি পৃথক `Symbol('id')` তৈরি করে প্রমাণ করো যে তারা কখনো সমান হয় না।

#### PRACTICE 2 MODEL SOLUTION

✓ STANDARD PATTERN

```
const id1 = Symbol("id");
const id2 = Symbol("id");

console.log("id1 === id2:", id1 === id2);
console.log("id1 description:", id1.description);
console.log("id2 description:", id2.description);
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
id1 === id2: false
id1 description: id
id2 description: id
```

### Practice 3: Global Symbol Registry with Symbol.for()

BEGINNER

**Problem Requirement:** `Symbol.for('app.user')` দিয়ে একটি গ্লোবাল সিম্বল তৈরি করে `Symbol.keyFor()` দিয়ে তার রেজিস্ট্রি কি রিট্রিভ করো।

#### PRACTICE 3 MODEL SOLUTION

✓ STANDARD PATTERN

```
const globalSym1 = Symbol.for("app.user");
const globalSym2 = Symbol.for("app.user");

console.log("Global Symbols Same?", globalSym1 === globalSym2);
console.log("Registry Key:", Symbol.keyFor(globalSym1));
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
Global Symbols Same? true
Registry Key: app.user
```

#### Practice 4: Custom Iterable with Symbol.iterator

INTERMEDIATE

**Problem Requirement:** একটি অবজেক্ট তৈরি করো যাতে `[Symbol.iterator]` মেথড ইমপ্লিমেন্ট করে সরাসরি `for ... of` দিয়ে উপাদান পাওয়া যায়।

##### PRACTICE 4 MODEL SOLUTION

✓ STANDARD PATTERN

```
const collection = {
  items: ["JavaScript", "TypeScript", "Node.js"],
  *[Symbol.iterator]() {
    for (const item of this.items) {
      yield item;
    }
  }
};

for (const tech of collection) {
  console.log(tech);
}
```

##### ► CONSOLE OUTPUT

OUTPUT

```
JavaScript
TypeScript
Node.js
```

#### Practice 5: Proxy Age Validation

INTERMEDIATE

**Problem Requirement:** একটি প্রক্সি তৈরি করো যা `age` প্রোপার্টিতে শুধু পজিটিভ সংখ্যা লিখতে দেবে, অন্যথায় `TypeError` থ্রো করবে।

##### PRACTICE 5 MODEL SOLUTION

✓ STANDARD PATTERN

```
const person = {};

const validator = new Proxy(person, {
  set(target, prop, value) {
    if (prop === "age") {
      if (typeof value !== "number" || value < 0) {
        throw new TypeError("Age must be a positive number!");
      }
    }
    target[prop] = value;
    return true;
  }
});

validator.age = 25;
console.log("Valid Age:", validator.age);
// validator.age = -5; // Throws TypeError
```

##### ► CONSOLE OUTPUT

OUTPUT

```
Valid Age: 25
```

#### Practice 6: Proxy Property Access Logger

INTERMEDIATE

**Problem Requirement:** একটি প্রক্সি তৈরি করো যা প্রতিটি প্রোপার্টি পড়া (GET) এবং লেখার (SET) সময় কনসোলে স্বয়ংক্রিয়ভাবে লগ তৈরি করবে।

##### PRACTICE 6 MODEL SOLUTION

✓ STANDARD PATTERN

```
const user = { name: "Ripon" };

const loggedUser = new Proxy(user, {
  get(target, prop) {
    console.log(`GET → ${String(prop)}`);
    return Reflect.get(target, prop);
  },
  set(target, prop, value) {
    console.log(`SET → ${String(prop)} = ${value}`);
    return Reflect.set(target, prop, value);
  }
});

const currentName = loggedUser.name;
loggedUser.age = 28;
```

##### ► CONSOLE OUTPUT

OUTPUT

```
GET → name
SET → age = 28
```

## Practice 7: Read-Only Object Proxy with Reflect

ADVANCED

**Problem Requirement:** একটি `createReadOnly(target)` ফাংশন বানাও যা অবজেক্টের কোনো প্রোপার্টি পরিবর্তন বা ডিলিট করতে দেবে না।

### PRACTICE 7 MODEL SOLUTION

✓ STANDARD PATTERN

```
function createReadOnly(target) {
  return new Proxy(target, {
    set(t, prop) {
      console.warn(`Cannot set '${String(prop)}': Object is read-only!`);
      return false;
    },
    deleteProperty(t, prop) {
      console.warn(`Cannot delete '${String(prop)}': Object is read-only!`);
      return false;
    }
  });
}

const config = createReadOnly({ apiEndpoint: "https://api.example.com" });
config.apiEndpoint = "https://hacked.com"; // Warning logged
console.log("Safe Config:", config.apiEndpoint);
```

### ▶ CONSOLE OUTPUT

OUTPUT

```
Cannot set 'apiEndpoint': Object is read-only!
Safe Config: https://api.example.com
```

## Practice 8: Role-Based Access Control (RBAC) Proxy

ADVANCED

**Problem Requirement:** এমন একটি প্রক্সি তৈরি করো যেখানে `user` রোল শুধু `name` এবং `admin` রোল `salary` দেখতে পারবে।

### PRACTICE 8 MODEL SOLUTION

✓ STANDARD PATTERN

```
function createSecureAccount(data, role) {
  return new Proxy(data, {
    get(target, prop) {
      if (prop === "salary" && role !== "admin") {
        return "🚫 Access Denied: Admin role required";
      }
      return Reflect.get(target, prop);
    }
  });
}

const employee = { name: "Anisur Rahman", salary: 85000 };
const userView = createSecureAccount(employee, "user");
const adminView = createSecureAccount(employee, "admin");

console.log("User views name:", userView.name);
console.log("User views salary:", userView.salary);
console.log("Admin views salary:", adminView.salary);
```

### ▶ CONSOLE OUTPUT

OUTPUT

```
User views name: Anisur Rahman
User views salary: 🚫 Access Denied: Admin role required
Admin views salary: 85000
```

## Practice 9: Integrated Metaprogramming Architecture

ADVANCED

**Problem Requirement:** একটি কাস্টম ক্লাস তৈরি করো যাতে `Symbol.iterator`, `Proxy` এবং `Reflect` তিনটি কনসেপ্ট একসাথে যুক্ত থাকবে।

### PRACTICE 9 MODEL SOLUTION

✓ STANDARD PATTERN

```
class ObservableStore {
  constructor(initialData = {}) {
    this.data = initialData;

    // Wrap internal data in proxy using Reflect
    return new Proxy(this, {
      set(target, prop, value) {
        console.log(`[Store Event] Mutated '${String(prop)}' to:`, value);
        return Reflect.set(target, prop, value);
      },
    });
  }

  *[Symbol.iterator]() {
    for (const [key, val] of Object.entries(this.data)) {
      yield { key, val };
    }
  }
}

const store = new ObservableStore({ theme: "dark", lang: "bn" });
store.version = "1.0.0";

for (const entry of store) {
  console.log(`Iterated: ${entry.key} = ${entry.val}`);
}
```

### ► CONSOLE OUTPUT

OUTPUT

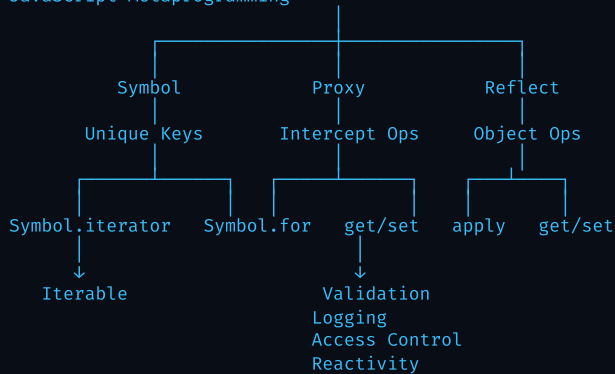
```
[Store Event] Mutated 'version' to: 1.0.0
Iterated: theme = dark
Iterated: lang = bn
```

## 28.Map 🧠 Final Mental Map & Metaprogramming Architecture

JavaScript Metaprogramming-এর ৩টি স্তর—Symbol, Proxy ও Reflect-এর সমন্বিত আর্কিটেকচার ম্যাপ:

### 📌 METAPROGRAMMING ARCHITECTURE TREE (SYMBOL, PROXY, REFLECT)

#### JavaScript Metaprogramming



### ★ এক নজরে ৩টি প্রধান স্তরের ভূমিকা

#### 1 Symbol

“আমার একটি অনন্য (Unique) কী বা অন্তর্নিহিত স্পেশাল প্রটোকল বিবেচ্যার দরকার।”

#### 2 Proxy

“কেউ অবজেক্টের সাথে কী অপারেশন করছে (get, set, has), তা ইন্টারসেপ্ট করে কাস্টম লজিক চালাতে চাই।”

#### 3 Reflect

“ইন্টারসেপ্ট করা অপারেশনটিকে স্ট্যান্ডার্ড ও পরিষ্কারভাবে মূল টার্গেট অবজেক্টের ওপর সম্পন্ন করতে চাই।”

### 🔗 Next Chapter Preview:

#### Chapter 29 — JavaScript Execution Model & Event Loop

পরবর্তী চ্যাপ্টারে আমরা শিখব Execution Context, Call Stack, Memory Heap, Microtask Queue, Macrotask Queue, Web APIs এবং Asynchronous Order of Execution।